

Contract Review — Developer Documentation

Version: 0.1.0 · **Last updated:** July 2026 Audience: Backend engineers, frontend engineers, ML/AI engineers

Table of Contents

1. [System Overview](#)
 2. [System Architecture Diagram](#)
 3. [Tech Stack](#)
 4. [Directory Structure](#)
 5. [Backend — FastAPI](#)
 6. [Agent Pipeline — LangGraph](#)
 7. [LLM Router](#)
 8. [Document Ingestion & RAG](#)
 9. [Frontend — React + Vite](#)
 10. [API Reference](#)
 11. [Data Models](#)
 12. [Environment Variables](#)
 13. [Local Development Setup](#)
 14. [Known Issues & Fixes](#)
-

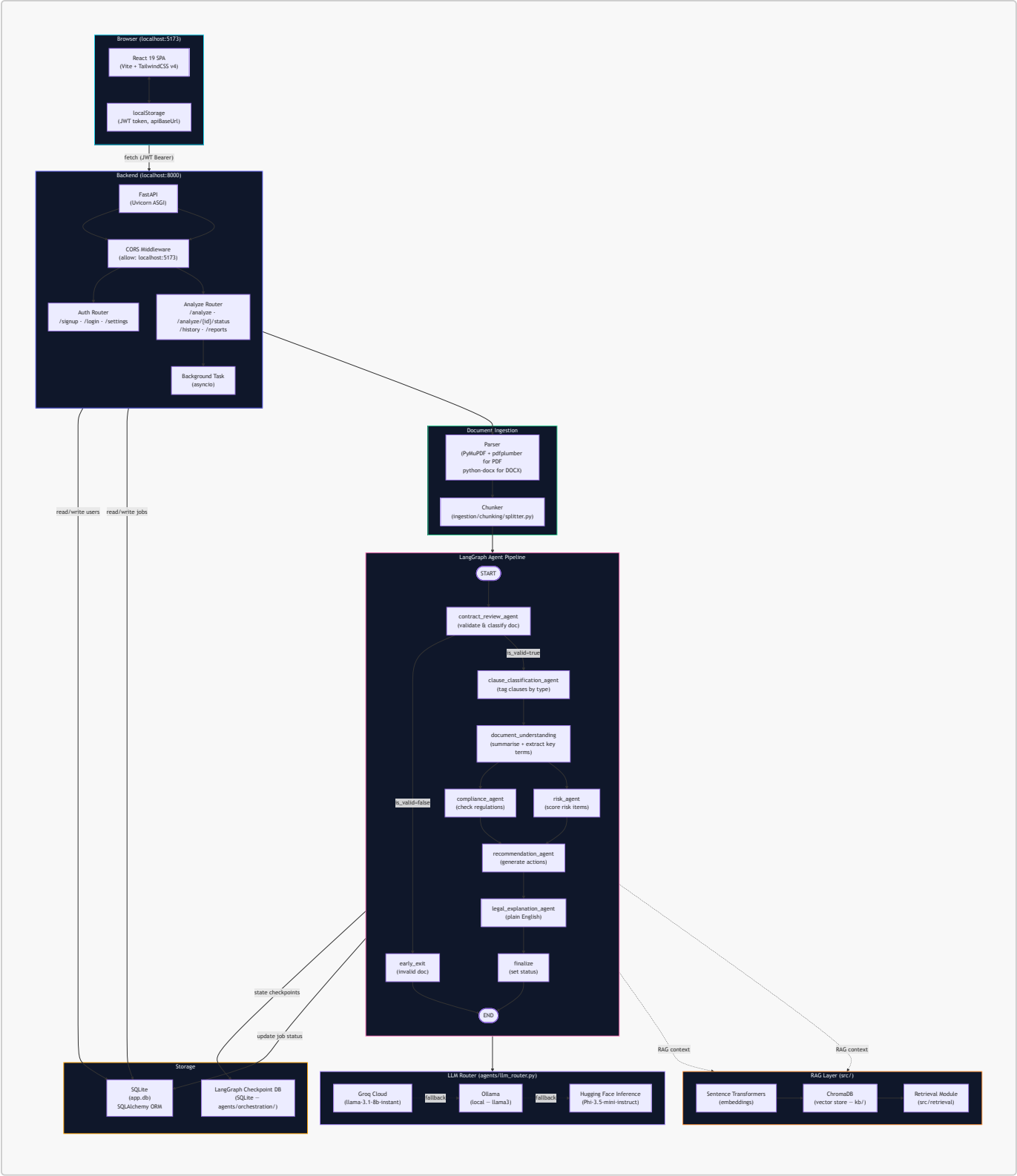
1. System Overview

Contract Review is a **multi-agent AI application** that automatically reviews employment/legal contracts. A user uploads a PDF or DOCX file; the backend parses, chunks, and passes it through a sequential **LangGraph** agent graph that performs clause classification, compliance checking, risk scoring, recommendation generation, and plain-English legal explanation. Results are persisted in SQLite and polled by the React frontend.

Key design decisions:

- Agents are **stateless functions** wired together by LangGraph — easy to add/remove/reorder.
 - LLM calls are routed through a **priority waterfall**: Groq Cloud → Ollama (local) → HuggingFace — so the system works with or without a cloud API key.
 - The backend uses **FastAPI background tasks** (not Celery) to keep infrastructure simple.
 - The frontend polls the status endpoint every 3 s rather than using WebSockets.
-

2. System Architecture Diagram



3. Tech Stack

Backend

Layer	Technology	Version	Notes
Web framework	FastAPI	≥ 0.115	Async, auto-docs at /docs
ASGI server	Uvicorn	≥ 0.32	[standard] extras for WebSocket support

Layer	Technology	Version	Notes
Agent orchestration	LangGraph	≥ 1.2.9	StateGraph with async nodes
Agent checkpointing	langgraph-checkpoint-sqlite	≥ 3.1	Persists mid-run state
LLM (primary)	Groq Cloud	API	llama-3.1-8b-instant — fastest, free tier
LLM (local fallback)	Ollama	—	llama3 running on localhost:11434
LLM (cloud fallback)	HuggingFace Inference	API	microsoft/Phi-3.5-mini-instruct
HTTP client	httpx	≥ 0.27	Async, used by LLM router
PDF parsing	PyMuPDF + pdfplumber	≥ 1.24 / 0.11	Dual-parser for best text extraction
DOCX parsing	python-docx	≥ 1.1	Handles .docx / .docm
Vector store	ChromaDB	≥ 0.5	Persisted locally in kb/
Embeddings	sentence-transformers	≥ 3.0	Local embedding model
ORM	SQLAlchemy	≥ 2.0	Declarative models
Database	SQLite	built-in	app.db — upgrade to Postgres for prod
Auth	python-jose + passlib[bcrypt]	≥ 3.3 / 1.7	JWT HS256, bcrypt password hashing
Validation	Pydantic	≥ 2.0	Request/response schemas
Config	python-dotenv	≥ 1.0	Loads .env at startup

Frontend

Layer	Technology	Version	Notes
UI framework	React	19.x	Concurrent mode
Build tool	Vite	8.x	HMR, dev proxy to FastAPI
Routing	react-router-dom	7.x	BrowserRouter with nested routes
Styling	TailwindCSS	4.x	Via @tailwindcss/vite plugin
Component library	shadcn/ui + Base UI	3.x / 1.x	Headless + styled primitives
Icons	lucide-react	1.x	—
Font	Geist Variable	—	Via @fontsource-variable/geist
Linting	ESLint + oxlint	8.x / 1.x	Dual linter setup

4. Directory Structure

```
Contract-Review-dev/  
├─ main.py # FastAPI app entrypoint, CORS, router registration  
├─ requirements.txt # Python dependencies
```

- └─ app.db # SQLite database (users, analysis jobs)
- └─ .env # ← YOU must create this (see §12)
- └─ api/ # FastAPI route handlers
 - └─ analyze.py # /analyze, /analyze/{id}/status, /history, /reports
 - └─ auth.py # /signup, /login, /settings (JWT auth)
 - └─ database.py # SQLAlchemy engine + session factory
 - └─ models.py # ORM models: User, AnalysisJob
- └─ agents/ # LLM agent implementations
 - └─ llm_router.py # Groq → Ollama → HF waterfall
 - └─ contract_review_agent.py # Gate: validates the doc is a contract
 - └─ clause_classification_agent.py # Tags each clause by type
 - └─ document_understanding_agent.py # Summarises doc, extracts key terms
 - └─ compliance_agent.py # Checks against regulatory frameworks
 - └─ risk_agent.py # Scores risk per clause
 - └─ recommendation_agent.py # Generates actionable recommendations
 - └─ legal_explanation_agent.py # Plain-English explanations
 - └─ schemas.py # Pydantic schemas shared by agents
 - └─ orchestration/
 - └─ orchestration.py # LangGraph StateGraph definition
 - └─ orchestration.md # Agent graph design notes
- └─ ingestion/ # Document parsing + chunking
 - └─ parsers/
 - └─ pdf_parser.py # PyMuPDF + pdfplumber
 - └─ docx_parser.py # python-docx
 - └─ chunking/
 - └─ splitter.py # Splits ParseResult into text chunks
- └─ src/ # RAG infrastructure
 - └─ embeddings/ # Sentence-transformer wrapper
 - └─ vectordb/ # ChromaDB client
 - └─ retrieval/ # Query + re-rank
 - └─ ingestion/ # KB ingestion pipeline
 - └─ metadata/ # Document metadata store
- └─ kb/ # ChromaDB persisted vector data
- └─ schemas/
 - └─ api_models.py # Shared Pydantic API response models
- └─ frontend/
 - └─ frontend/ # React application root
 - └─ vite.config.js # Build config + dev proxy to :8000
 - └─ src/
 - └─ App.jsx # Router setup
 - └─ lib/api.js # All fetch() calls to the backend
 - └─ hooks/useLocalStorage.js
 - └─ pages/
 - └─ AnalysisPage.jsx # Upload + polling UI
 - └─ LoginPage.jsx
 - └─ SignupPage.jsx
 - └─ HistoryPage.jsx
 - └─ ReportsPage.jsx
 - └─ PreviewPage.jsx
 - └─ SettingsPage.jsx # API base URL config
 - └─ components/
 - └─ AppShell.jsx # Sidebar layout wrapper
 - └─ UploadZone.jsx # Drag-and-drop file input

5. Backend — FastAPI

Entry point — `main.py`

```
load_dotenv()
├─ api.models → create_all() on SQLite
├─ CORS middleware (allow_origins: localhost:5173)
├─ include_router(auth.router)
├─ include_router(analyze.router)
└─ GET /health
```

[!IMPORTANT] `load_dotenv()` runs **before** all other imports because `agents/llm_router.py` reads `GRQO_API_KEY` at import time. Import order in `main.py` must not be changed.

Authentication — `api/auth.py`

- `POST /signup` — JSON body `{email, password}` → bcrypt hash → store User → `201 UserResponse`
- `POST /login` — **form-encoded** `username + password` (OAuth2 spec) → JWT HS256 → `Token`
- `GET /settings` — Bearer token → current `UserResponse`
- Tokens expire in **7 days** by default (`ACCESS_TOKEN_EXPIRE_MINUTES = 60*24*7`)

Analysis Jobs — `api/analyze.py`

Step	Detail
1. Validate extension	<code>.pdf, .docx, .docm</code> only
2. Write to temp file	<code>tempfile.NamedTemporaryFile</code>
3. Create DB job	<code>status = "pending"</code>
4. Queue background task	<code>BackgroundTasks.add_task(process_contract_background)</code>
5. Return immediately	<code>{contract_id, status: "pending"}</code>
6. Background: parse	<code>get_parser() → parser.parse()</code>
7. Background: chunk	<code>chunk_parse_result()</code>
8. Background: agents	<code>run_contract_review(state, thread_id=contract_id)</code>
9. Background: update DB	<code>_update_job_status(contract_id, status, result)</code>

6. Agent Pipeline — LangGraph

The graph is built once and compiled with an **AsyncSqliteSaver** checkpointer so that intermediate state is preserved across agent steps.

Execution Flow

```
START
├─ contract_review_agent ← Validates the document is a contract
├─ is_valid=False → early_exit → END (no further processing)
├─ is_valid=True → clause_classification_agent
└─ document_understanding (async)
```

```

├─ compliance_agent ─┐
├─ risk_agent ─────────┘
▼
recommendation_agent
├─ legal_explanation_agent
├─
finalize → END

```

ContractReviewState — shared TypedDict

Field	Type	Written by
contract_id	str	API layer
raw_text	str	API layer
chunks	List[str]	API layer
file_metadata	Dict	API layer
contract_metadata	Dict	contract_review_agent
is_valid_contract	bool	contract_review_agent
classified_clauses	List[Dict]	clause_classification_agent
document_summary	str	document_understanding_agent
key_terms	Dict	document_understanding_agent
retrieved_context	List[Dict]	RAG retrieval
compliance_results	Dict	compliance_agent
risk_results	Dict	risk_agent
recommendations	List[Dict]	recommendation_agent
legal_explanations	List[Dict]	legal_explanation_agent
errors	List[Dict]	All agents (append-only)
completed_steps	List[str]	All agents (append-only)
status	Literal	finalize / early_exit

Error Handling

Every agent node is wrapped in `_safe_call()`. If an agent raises an exception, it is caught, logged, appended to `state["errors"]`, and execution continues. The final status will be `completed_with_errors` instead of `completed`.

7. LLM Router

File: `agents/llm_router.py`

All agents call `await chat(messages, format="json")` — they never import a specific provider.

```
chat(messages)
1. Try Groq Cloud (GROQ_API_KEY required – llama-3.1-8b-instant)
  ↓ exception
2. Try Ollama (requires local daemon on port 11434 – llama3)
  ↓ exception
3. Try HuggingFace (HF_API_TOKEN optional – Phi-3.5-mini-instruct)
  ↓ exception
4. Return "{}" / empty string (agents handle gracefully)
```

`chat_json()` wraps `chat()` to parse and return a Python dict/list. The `StructuredLLM` class provides a `get_structured_llm(schema)` compatibility shim for agents that expect a Pydantic-schema-based LLM.

8. Document Ingestion & RAG

Parsing

Format	Primary parser	Fallback
<code>.pdf</code>	PyMuPDF (<code>fitz</code>)	pdfplumber
<code>.docx</code> / <code>.docm</code>	python-docx	—

All parsers return a `ParseResult(full_text, metadata)` object.

Chunking — `ingestion/chunking/splitter.py`

`chunk_parse_result(parsed, source_name)` splits `full_text` into `Chunk` objects. Chunks are passed as `state["chunks"]` to the agent graph.

RAG — `src/`

Module	Role
<code>src/embeddings/</code>	Wraps <code>sentence-transformers</code> to produce vectors
<code>src/vectordb/</code>	ChromaDB client; persists to <code>kb/</code>
<code>src/ingestion/</code>	Offline KB ingestion (run via <code>build_kb.sh</code>)
<code>src/retrieval/</code>	Similarity search called by <code>document_understanding_agent</code> and <code>compliance_agent</code>

[NOTE] Run `./build_kb.sh` to populate the ChromaDB knowledge base before starting the backend. Without this, RAG context will be empty and agents will rely solely on their LLM prompt.

9. Frontend — React + Vite

Routing (App.jsx)

```
/ → HomePage
/analysis → AnalysisPage (upload + poll)
/preview → PreviewPage (contract text view)
/reports → ReportsPage (completed reports)
/history → HistoryPage (all jobs)
/login → LoginPage
```

```
/signup → SignupPage  
/settings → SettingsPage (API base URL, theme)
```

All routes are wrapped in `<AppShell />` which renders the sidebar nav.

API Client — `src/lib/api.js`

All network calls go through this module. Pages pass `apiBaseUrl` (from `useLocalStorage`) as a parameter so the user can point the frontend at any backend URL from the Settings page.

Function	Method	Endpoint
<code>startAnalysis(file, baseUrl)</code>	POST	<code>/analyze</code>
<code>checkAnalysisStatus(id, baseUrl)</code>	GET	<code>/analyze/{id}/status</code>
<code>fetchReports(baseUrl)</code>	GET	<code>/reports</code>
<code>fetchHistory(baseUrl)</code>	GET	<code>/history</code>
<code>mapAnalysisResponse(raw)</code>	—	Client-side transform
<code>generateReportBlob(results, meta)</code>	—	Client-side <code>.txt</code> export

Auth Flow

```
POST /login (form-encoded)  
└─ stores token → localStorage['token']  
└─ stores {email} → localStorage['currentUser']  
  
getAuthHeaders() ← reads localStorage['token']  
└─ { Authorization: 'Bearer <token>' }  
  
All protected requests attach this header automatically.
```

Vite Dev Proxy (vite.config.js)

The following routes are proxied from `localhost:5173` to `localhost:8000` so the browser never makes a cross-origin request during development:

```
/analyze /login /signup /settings /history /reports /health
```

10. API Reference

Auth

Method	Path	Auth	Request	Response
POST	<code>/signup</code>		<code>{email, password}</code> JSON	<code>{id, email}</code>
POST	<code>/login</code>		form-encoded <code>username + password</code>	<code>{access_token, token_type}</code>
GET	<code>/settings</code>	Bearer	—	<code>{id, email}</code>

Analysis

Method	Path	Auth	Request	Response
POST	/analyze	Bearer	multipart file	{contract_id, status: "pending"}
GET	/analyze/{id}/status	Bearer	—	Status object (see below)
GET	/history	Bearer	—	Array<Job>
GET	/reports	Bearer	—	Array<Job> (completed only)

Health

Method	Path	Auth	Response
GET	/health		{status: "ok"}

Status Response Schema

```
{
  "contract_id": "contract-abc123",
  "status": "pending | in_progress | completed | completed_with_errors | failed",

  // Only present when status ∈ {completed, completed_with_errors, failed}:
  "contract_metadata": { ... },
  "compliance_results": {
    "violations": [{ "regulation", "clause_id", "severity", "description" }],
    "missing_requirements": [ "..." ]
  },
  "risk_results": {
    "overall_risk_score": 14, // 0-(categoryCount × 9)
    "category_risks": { ... },
    "risk_items": [{ "risk_type", "likelihood", "impact", "description" }]
  },
  "recommendations": [{ ... }],
  "legal_explanations": [{ ... }],
  "errors": [{ "agent", "error", "timestamp" }],
  "completed_steps": ["contract_review_agent", "clause_classification_agent", ...]
}
```

11. Data Models

SQLAlchemy (api/models.py)

User

Column	Type	Notes
id	Integer PK	Auto-increment
email	String UNIQUE	Login identifier
hashed_password	String	bcrypt

AnalysisJob

Column	Type	Notes
id	Integer PK	—
user_id	FK → User.id	—
contract_id	String UNIQUE	contract-{hex12}
filename	String	Original upload name
status	String	pending / in_progress / completed / completed_with_errors / failed
result_data	JSON	Full agent pipeline output
created_at	DateTime	Auto now()

12. Environment Variables

Create a `.env` file in the project root (never commit it):

```
# — LLM Providers —————
GROQ_API_KEY=gsk_... # Primary LLM – get free key at console.groq.com
GROQ_MODEL=llama-3.1-8b-instant # Override default model

OLLAMA_BASE_URL=http://localhost:11434 # Local Ollama daemon URL
OLLAMA_MODEL=llama3 # Model pulled in Ollama

HF_API_TOKEN=hf_... # Optional – HuggingFace token for higher rate limits
HF_MODEL=microsoft/Phi-3.5-mini-instruct

# — Auth —————
JWT_SECRET=change-me-to-a-long-random-string # REQUIRED in production

# — App —————
# No additional variables required for local SQLite setup
```

[!CAUTION] The default `JWT_SECRET` is `"super-secret-default-key-change-in-prod"`. **Never deploy without setting a strong secret.** All existing tokens become invalid when the secret changes.

13. Local Development Setup

Prerequisites

- Python 3.11+
- Node.js 20+
- (Optional) [Ollama](#) for local LLM fallback
- A [Groq API key](#) for the primary LLM

Backend

```
# 1. Create and activate virtual environment
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
```

```
# 2. Install dependencies
pip install -r requirements.txt

# 3. Create .env (see §12)
cp .env.example .env # then fill in your keys

# 4. (Optional) Build the RAG knowledge base
./build_kb.sh

# 5. Start the server
uvicorn main:app --reload --port 8000
# Auto-docs available at http://localhost:8000/docs
```

Frontend

```
cd frontend/frontend

# Install dependencies (first time only)
npm install

# Start the dev server (proxies API calls to :8000)
npm run dev
# App available at http://localhost:5173
```

Ollama (optional — local LLM fallback)

```
# Install Ollama, then pull the model
ollama pull llama3

# Ollama runs automatically on localhost:11434
```

14. Known Issues & Fixes

#	Severity	Component	Issue	Fix Applied
1	High	vite.config.js	No dev proxy — browser makes cross-origin requests to :8000, triggering CORS errors	Proxy added for all API routes
2	High	SignupPage.jsx	Password field rendered before email field — breaks autofill and confuses users	Field order corrected
3	Medium	main.py	ALLOWED_ORIGINS only covers localhost:5173 — any other origin (staging, production) will be CORS-blocked	Add production URL to ALLOWED_ORIGINS before deploying
4	Medium	api/auth.py	JWT_SECRET defaults to a known weak string — all instances share the same secret if .env is not set	Set JWT_SECRET in .env
5	Medium	agents/orchestration	knowledge_retrieval_agent is commented out in	Uncomment + implement when KB is

#	Severity	Component	Issue	Fix Applied
			<code>document_understanding_node</code> — RAG context is not injected	ready
6	Low	<code>api/analyze.py</code>	<code>risk_score</code> in <code>/history</code> and <code>/reports</code> reads <code>result_data.risk_results.risk_score</code> but the actual key is <code>overall_risk_score</code> — always returns <code>0</code>	Update key to <code>overall_risk_score</code>

Generated from codebase analysis of `Contract-Review-dev` — July 2026